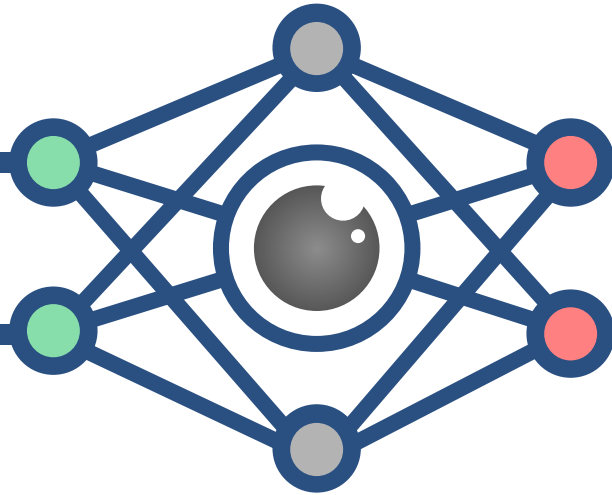


CS3485

Deep Learning for Computer Vision



Lec 7: Convolutional Neural Networks

Announcements

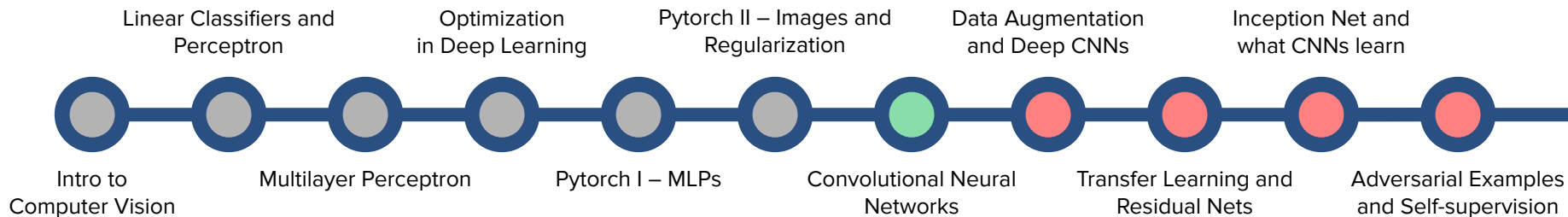
- Lab3:
 - It is out! It will be due next Friday.
 - Things may get slower with the experiments. Make sure to use GPUs,
 - Start earlier rather than later due to errors and bugs
- A great resource for last lecture: 3Brow1Blue [videos on Deep Learning!](#)
- Also, check out the video on the right!
- And let's finish the previous lecture.

First colored/RGB images!

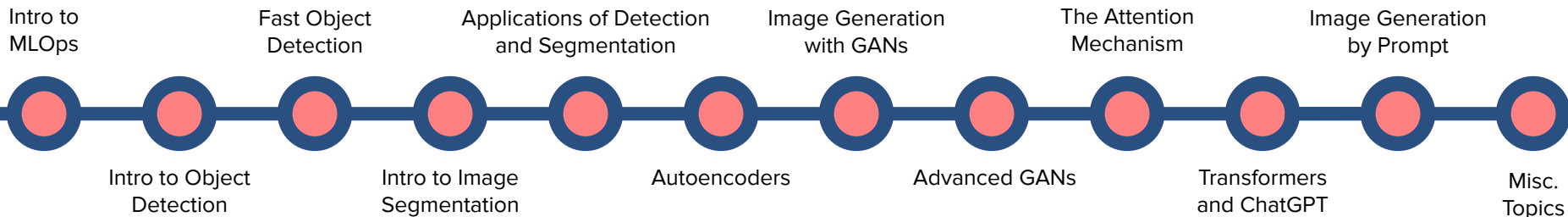


(Tentative) Lecture Roadmap

Basics of Deep Learning



Deep Learning and Computer Vision in Practice



Fashion MNIST

- Last time, we used the MNIST dataset to try out our Multilayer Perceptron (MLP) using PyTorch.
- Today, we'll try a more challenging Dataset called **Fashion MNIST (FMNIST)**.
- It follows the same specs as MNIST: there are *70k* images (*60k* for training and *10k* for testing) of size *28×28* of *10* classes.
- Instead of handwritten digits, the data now is of clothing articles and the classes are: *T-shirt/top*, *Trouser*, *Pullover*, *Dress*, *Coat*, *Sandal*, *Shirt*, *Sneaker*, *Bag*, *Ankle boot*.
- Let's see how our previous network performs in this new data and take the time to review last class.



Loading the Data and DataLoaders

- Let's start by downloading the dataset:

```
import torch
from torchvision import datasets
from torch.utils.data import Dataset, DataLoader

device = 'cuda' if torch.cuda.is_available() else 'cpu'
fmnist_train = datasets.FashionMNIST('~/.data/FMNIST', download=True, train=True)
fmnist_test = datasets.FashionMNIST('~/.data/FMNIST', download=True, train=False)
x_train, y_train = fmnist_train.data, fmnist_train.targets
x_test, y_test = fmnist_test.data, fmnist_test.targets

class FMNISTDataset(Dataset):
    def __init__(self, x, y):
        x = x.view(-1, 28*28).float()/255
        self.x, self.y = x, y
    def __getitem__(self, ix):
        return self.x[ix].to(device), self.y[ix].to(device)
    def __len__(self):
        return len(self.x)

train_dataset = FMNISTDataset(x_train, y_train)
train_dl = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_dataset = FMNISTDataset(x_test, y_test)
test_dl = DataLoader(test_dataset, batch_size=32, shuffle=True)
```

Loading the Data and DataLoaders

- Let's start by downloading the dataset:

```
import torch
from torchvision import datasets
from torch.utils.data import Dataset, DataLoader
```

Detects the GPU

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

```
fmnist_train = datasets.FashionMNIST('~/.data/FMNIST', download=True, train=True)
fmnist_test = datasets.FashionMNIST('~/.data/FMNIST', download=True, train=False)
x_train, y_train = fmnist_train.data, fmnist_train.targets
x_test, y_test = fmnist_test.data, fmnist_test.targets
```

Loads the Fashion MNIST training and testing data.

```
class FMNISTDataset(Dataset):
    def __init__(self, x, y):
        x = x.view(-1, 28*28).float()/255
        self.x, self.y = x, y
    def __getitem__(self, ix):
        return self.x[ix].to(device), self.y[ix].to(device)
    def __len__(self):
        return len(self.x)
```

Creates a Dataset class that preprocesses the data (reshapes and rescales it).

```
train_dataset = FMNISTDataset(x_train, y_train)
train_dl = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_dataset = FMNISTDataset(x_test, y_test)
test_dl = DataLoader(test_dataset, batch_size=32, shuffle=True)
```

Define the Dataloaders, that coordinate how the data will be read.

Visualizing the data

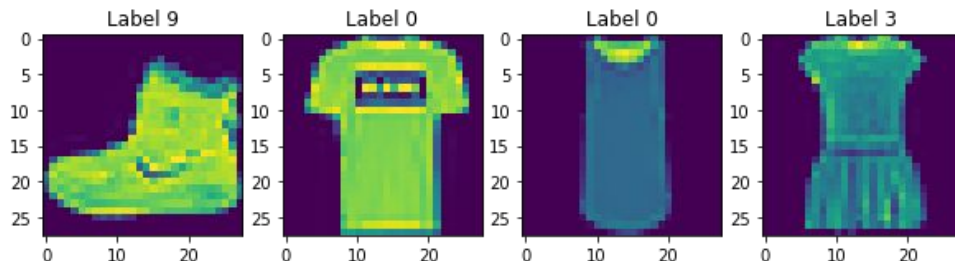
- We also check a bit of how the data looks like:

```
print(x_train.shape, y_train.shape)
print(x_test.shape, y_test.shape)
```

```
torch.Size([60000, 28, 28]) torch.Size([60000])
torch.Size([10000, 28, 28]) torch.Size([10000])
```

- Following the **(N, C, H, W)**, we know that
 - The training set has **60000** points (N), of 1 channel each (C), and each points has height **28** (H) and width **28** (W). The same for the testing set, but with **10000** samples (N).
 - There are **60000** scalar labels for training and **10000** for testing.
- We can furthermore read a few data points and their labels:

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10,3))
for i in range(4):
    plt.subplot(1,4,i+1)
    plt.imshow(x_train[i])
    plt.title(f"Label {y_train[i]}")
plt.show()
```



Creating the network and visualizing it

- Today, we'll use `nn.Sequential()` to create a NN of one hidden layer with 1k units:

```
import torch.nn as nn
model = nn.Sequential(nn.Linear(28 * 28, 1000), nn.ReLU(), nn.Linear(1000, 10)).to(device)
```

- We also introduce the `summary` function to visualize the network:

```
# In order to install torchsummary, run
# 'pip install torch summary'
from torchsummary import summary
summary(model, (1, 28*28))
```

where `(1, 28*28)` is the size of the model's input (matrices of size 1×784).

- Note that we need to learn almost *800k* weights!

```
-----
Layer (type)          Output Shape          Param #
-----
Linear-1              [-1, 1, 1, 1000]      785,000
ReLU-2                [-1, 1, 1, 1000]       0
Linear-3              [-1, 1, 1, 10]        10,010
=====
Total params: 795,010
Trainable params: 795,010
Non-trainable params: 0
-----
Input size (MB): 0.00
Forward/backward pass size (MB): 0.02
Params size (MB): 3.03
Estimated Total Size (MB): 3.05
-----
```

Defining the loss and the optimizer

- We also choose the Cross Entropy as our loss function and ADAM as our optimizer with *0.001* as the learning rate:

```
from torch.optim import Adam
loss_fn = nn.CrossEntropyLoss()
opt = Adam(model.parameters(), lr=1e-3)
```

- Like we did last time, we define two auxiliary functions: one to do all the steps for training and the other to compute classification accuracies.

```
def train_batch(x, y, model, opt, loss_fn):
    model.train()

    opt.zero_grad() # Flush memory
    batch_loss = loss_fn(model(x), y) # Compute loss
    batch_loss.backward() # Compute gradients
    opt.step() # Make a GD step

    return batch_loss.detach().cpu().numpy()
```

```
@torch.no_grad()
def accuracy(x, y, model):
    model.eval()

    prediction = model(x)
    argmaxes = prediction.argmax(dim=1)
    s = torch.sum((argmaxes == y).float())/len(y)

    return s.cpu().numpy()
```

Training the network

- Then, we train the network:

```
import numpy as np
losses, accuracies, n_epochs = [], []
n_epochs = 5
for epoch in range(n_epochs):
    print(f"Running epoch {epoch + 1} of {n_epochs}")

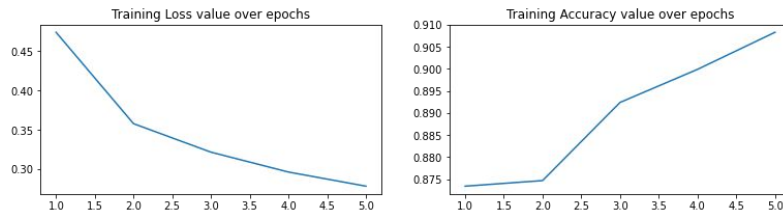
    epoch_losses, epoch_accuracies = [], []
    for batch in train_dl:
        x, y = batch
        batch_loss = train_batch(x, y, model, opt, loss_func)
        epoch_losses.append(batch_loss)
    epoch_loss = np.mean(epoch_losses)

    for batch in train_dl:
        x, y = batch
        batch_acc = accuracy(x, y, model)
        epoch_accuracies.append(batch_acc)
    epoch_accuracy = np.mean(epoch_accuracies)

    losses.append(epoch_loss)
    accuracies.append(epoch_accuracy)
```

- And visualize how it did during training:

```
import matplotlib.pyplot as plt
plt.figure(figsize=(13,3))
plt.subplot(121)
plt.title('Training Loss value over epochs')
plt.plot(np.arange(n_epochs) + 1, losses)
plt.subplot(122)
plt.title('Testing Accuracy value over epochs')
plt.plot(np.arange(n_epochs) + 1, accuracies)
```



- This learning procedure (of 800k weights) took around 43.6 s.

Testing the learned classifier

- Finally, we can test our classifier (*running again may give be slightly different results*):

```
epoch_accuracies = []
for batch in test_dl:
    x, y = batch
    batch_acc = accuracy(x, y, model)
    epoch_accuracies.append(batch_acc)

print(f"Test accuracy: {np.mean(epoch_accuracies)}")
```

```
Test accuracy: 0.8813897967338562
```

- When we did this same procedure with the same parameters on the MNIST digit dataset, **we got 96% testing accuracy**.
- This shows **how much harder Fashion MNIST is** and that we need more to use more techniques to improve its performance.
- How can we improve this testing result **without adding many more weights**?

The Convolution Operation

- A very important operation in our solution to better our performance is **Convolution**.
- Take two matrices $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{p \times q}$.
- The convolution between A and B , denoted here as $A \otimes B$ is another matrix, C , such that:

$$(A \otimes B)_{i,j} = C_{i,j} = \sum_{k=0}^{p-1} \sum_{l=0}^{q-1} A_{k,l} \times B_{i-k,j-l}$$

A	B	$A \otimes B$																													
<table><tr><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td>5</td><td>6</td><td>7</td><td>8</td></tr><tr><td>9</td><td>10</td><td>11</td><td>12</td></tr><tr><td>13</td><td>14</td><td>15</td><td>16</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	<table><tr><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td></tr></table>	1	2	3	4	<table><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>									
1	2	3	4																												
5	6	7	8																												
9	10	11	12																												
13	14	15	16																												
1	2																														
3	4																														

- Despite its apparent complexity, **it is quite simple**: we are simply “scanning” matrix A with windows of the size of matrix B and, as we scan, we multiply all elements of the window in A with B element wise, and finally sum the multiplication results up.
- In this case, the matrix B , which is sweeping over A , is called **kernel** or **filter** and the convolution process is sometimes called **filtering**.

The Convolution Operation

- A very important operation in our solution to better our performance is **Convolution**.
- Take two matrices $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{p \times q}$.
- The convolution between A and B , denoted here as $A \otimes B$ is another matrix, C , such that:

$$(A \otimes B)_{i,j} = C_{i,j} = \sum_{k=0}^{p-1} \sum_{l=0}^{q-1} A_{k,l} \times B_{i-k,j-l}$$

A	B	$A \otimes B$																													
<table><tr><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td>5</td><td>6</td><td>7</td><td>8</td></tr><tr><td>9</td><td>10</td><td>11</td><td>12</td></tr><tr><td>13</td><td>14</td><td>15</td><td>16</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	<table><tr><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td></tr></table>	1	2	3	4	<table><tr><td>44</td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>	44								
1	2	3	4																												
5	6	7	8																												
9	10	11	12																												
13	14	15	16																												
1	2																														
3	4																														
44																															

$$1 \times 1 + 2 \times 2 + 5 \times 3 + 6 \times 4 = 44$$

- Despite its apparent complexity, **it is quite simple**: we are simply “scanning” matrix A with windows of the size of matrix B and, as we scan, we multiply all elements of the window in A with B element wise, and finally sum the multiplication results up.
- In this case, the matrix B , which is sweeping over A , is called **kernel** or **filter** and the convolution process is sometimes called **filtering**.

The Convolution Operation

- A very important operation in our solution to better our performance is **Convolution**.
- Take two matrices $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{p \times q}$.
- The convolution between A and B , denoted here as $A \otimes B$ is another matrix, C , such that:

$$(A \otimes B)_{i,j} = C_{i,j} = \sum_{k=0}^{p-1} \sum_{l=0}^{q-1} A_{k,l} \times B_{i-k,j-l}$$

A				B		$A \otimes B$		
1	2	3	4	1	2	44	54	
5	6	7	8	3	4			
9	10	11	12					
13	14	15	16					

$$2 \times 1 + 3 \times 2 + 6 \times 3 + 7 \times 4 = 54$$

- Despite its apparent complexity, **it is quite simple**: we are simply “scanning” matrix A with windows of the size of matrix B and, as we scan, we multiply all elements of the window in A with B element wise, and finally sum the multiplication results up.
- In this case, the matrix B , which is sweeping over A , is called **kernel** or **filter** and the convolution process is sometimes called **filtering**.

The Convolution Operation

- A very important operation in our solution to better our performance is **Convolution**.
- Take two matrices $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{p \times q}$.
- The convolution between A and B , denoted here as $A \otimes B$ is another matrix, C , such that:

$$(A \otimes B)_{i,j} = C_{i,j} = \sum_{k=0}^{p-1} \sum_{l=0}^{q-1} A_{k,l} \times B_{i-k,j-l}$$

A				B		$A \otimes B$		
1	2	3	4	1	2	44	54	64
5	6	7	8	3	4			
9	10	11	12					
13	14	15	16					

$$3 \times 1 + 4 \times 2 + 7 \times 3 + 8 \times 4 = 64$$

- Despite its apparent complexity, **it is quite simple**: we are simply “scanning” matrix A with windows of the size of matrix B and, as we scan, we multiply all elements of the window in A with B element wise, and finally sum the multiplication results up.
- In this case, the matrix B , which is sweeping over A , is called **kernel** or **filter** and the convolution process is sometimes called **filtering**.

The Convolution Operation

- A very important operation in our solution to better our performance is **Convolution**.
- Take two matrices $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{p \times q}$.
- The convolution between A and B , denoted here as $A \otimes B$ is another matrix, C , such that:

$$(A \otimes B)_{i,j} = C_{i,j} = \sum_{k=0}^{p-1} \sum_{l=0}^{q-1} A_{k,l} \times B_{i-k,j-l}$$

A				B		$A \otimes B$		
1	2	3	4	1	2	44	54	64
5	6	7	8	3	4	84		
9	10	11	12					
13	14	15	16					

$$5 \times 1 + 6 \times 2 + 9 \times 3 + 10 \times 4 = 84$$

- Despite its apparent complexity, **it is quite simple**: we are simply “scanning” matrix A with windows of the size of matrix B and, as we scan, we multiply all elements of the window in A with B element wise, and finally sum the multiplication results up.
- In this case, the matrix B , which is sweeping over A , is called **kernel** or **filter** and the convolution process is sometimes called **filtering**.

The Convolution Operation

- A very important operation in our solution to better our performance is **Convolution**.
- Take two matrices $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{p \times q}$.
- The convolution between A and B , denoted here as $A \otimes B$ is another matrix, C , such that:

$$(A \otimes B)_{i,j} = C_{i,j} = \sum_{k=0}^{p-1} \sum_{l=0}^{q-1} A_{k,l} \times B_{i-k,j-l}$$

A				B		$A \otimes B$		
1	2	3	4	1	2	44	54	64
5	6	7	8	3	4	84	94	
9	10	11	12					
13	14	15	16					

$$6 \times 1 + 7 \times 2 + 10 \times 3 + 11 \times 4 = 94$$

- Despite its apparent complexity, **it is quite simple**: we are simply “scanning” matrix A with windows of the size of matrix B and, as we scan, we multiply all elements of the window in A with B element wise, and finally sum the multiplication results up.
- In this case, the matrix B , which is sweeping over A , is called **kernel** or **filter** and the convolution process is sometimes called **filtering**.

The Convolution Operation

- A very important operation in our solution to better our performance is **Convolution**.
- Take two matrices $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{p \times q}$.
- The convolution between A and B , denoted here as $A \otimes B$ is another matrix, C , such that:

$$(A \otimes B)_{i,j} = C_{i,j} = \sum_{k=0}^{p-1} \sum_{l=0}^{q-1} A_{k,l} \times B_{i-k,j-l}$$

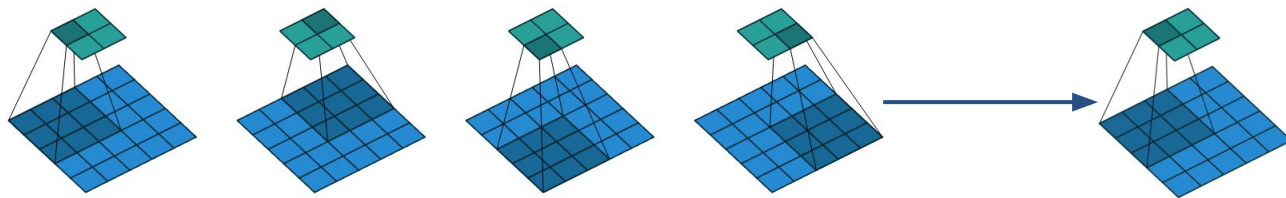
A	B	$A \otimes B$																													
<table><tr><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td>5</td><td>6</td><td>7</td><td>8</td></tr><tr><td>9</td><td>10</td><td>11</td><td>12</td></tr><tr><td>13</td><td>14</td><td>15</td><td>16</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	<table><tr><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td></tr></table>	1	2	3	4	<table><tr><td>44</td><td>54</td><td>64</td></tr><tr><td>84</td><td>94</td><td>104</td></tr><tr><td>124</td><td>134</td><td>144</td></tr></table>	44	54	64	84	94	104	124	134	144
1	2	3	4																												
5	6	7	8																												
9	10	11	12																												
13	14	15	16																												
1	2																														
3	4																														
44	54	64																													
84	94	104																													
124	134	144																													

And so on ...

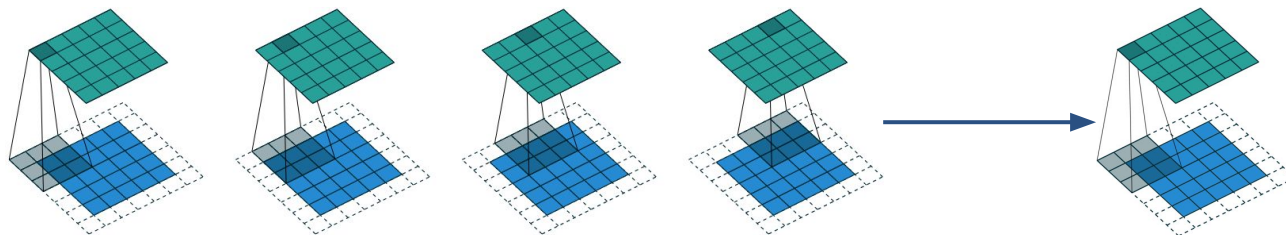
- Despite its apparent complexity, **it is quite simple**: we are simply “scanning” matrix A with windows of the size of matrix B and, as we scan, we multiply all elements of the window in A with B element wise, and finally sum the multiplication results up.
- In this case, the matrix B , which is sweeping over A , is called **kernel** or **filter** and the convolution process is sometimes called **filtering**.

Strides and paddings

- The final **convolved matrix** $A \otimes B$ has a size that depends on the sizes of A and B .
- In order to control the final size, we can change the convolution operation in two ways:
 - By changing the **convolutional stride**: the stride is the step you take when moving from one window to the next when sweeping. In usual convolution, the stride is **1**. Below, it is **2**:

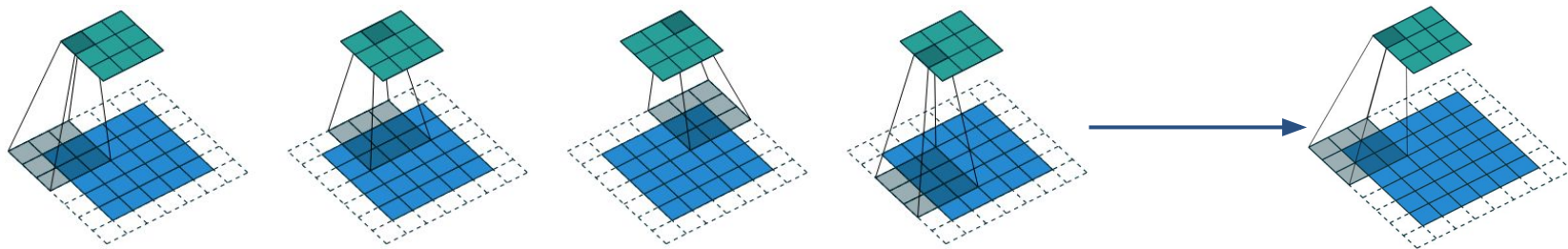


- Via **padding**: add a frame of zero valued entries around the matrix being swept by the kernel.



Strides and paddings

- The stride (skipping more entries as we sweep) and the amount of padding (the “thickness” of the frame) are then parameters of our convolution operation*.
- We can also have both strides and padding together**:



- There are other changes one can make in order **to increase** the size of the output matrix, compared to the input matrix.
- This operation is called a **Deconvolution operation**, which is crucial in many Deep Learning solutions for Computer Vision (*more on it later in the course*).

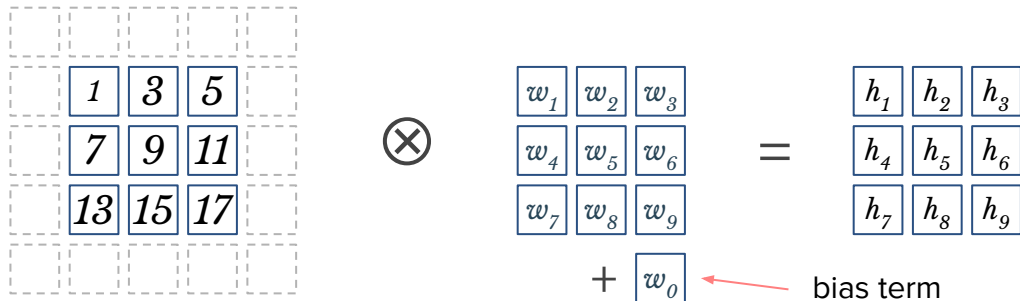
* All the animations for padding and strides were taken from this very pedagogical [paper](#) and [github account](#).

** [Here](#)'s one great visualization of the different combinations one could get.

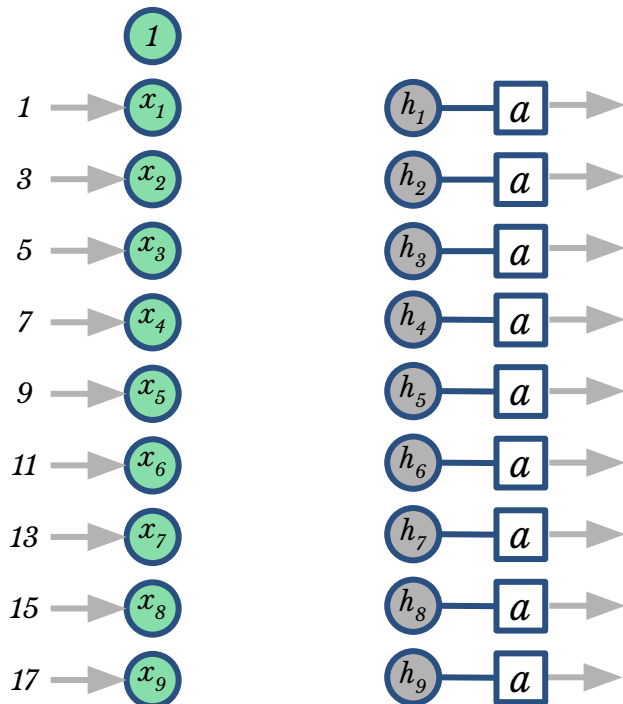
Convolution as a Neural Network Layer

- Convolutions can be implemented as a NN layer, called **Convolutional Layer (ConvLayer)**.
- In it, we only need to zero many of the connections from its input to output (according to the convolution step) and keep the weights the same across units.

Convolution Operation



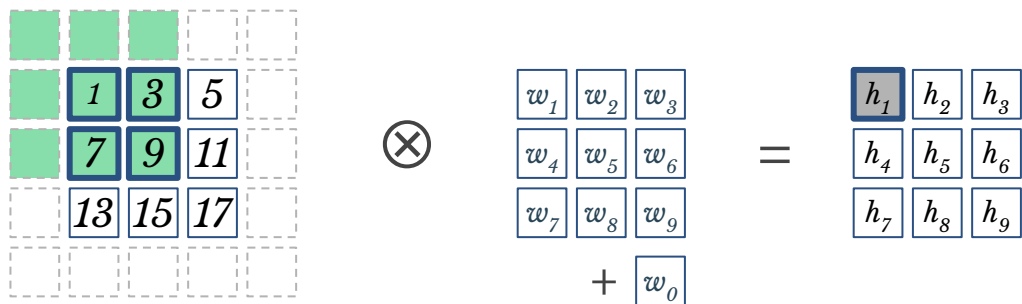
Convolutional Layer



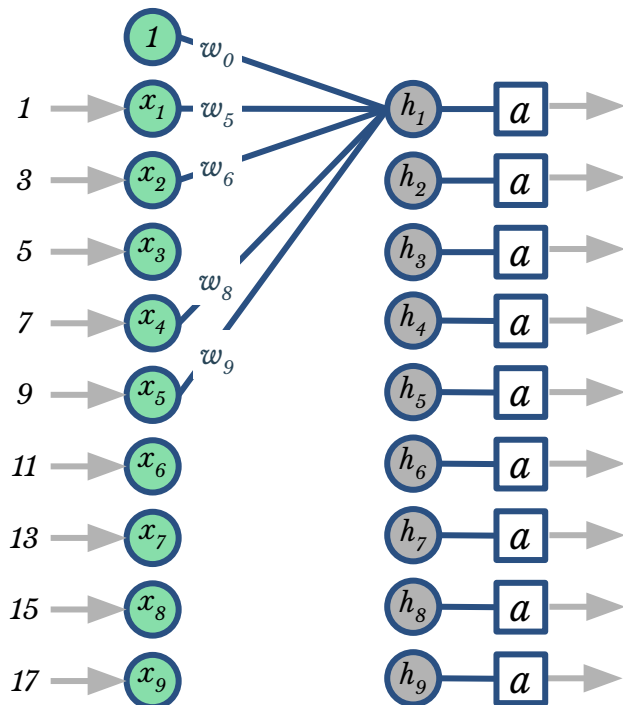
Convolution as a Neural Network Layer

- Convolutions can be implemented as a NN layer, called **Convolutional Layer (ConvLayer)**.
- In it, we only need to zero many of the connections from its input to output (according to the convolution step) and keep the weights the same across units.

Convolution Operation



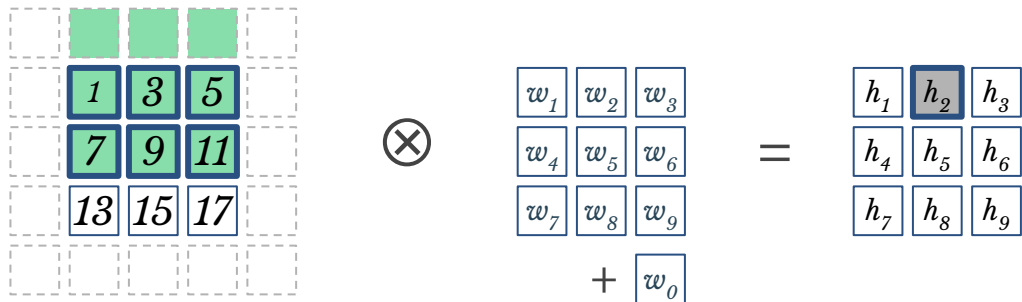
Convolutional Layer



Convolution as a Neural Network Layer

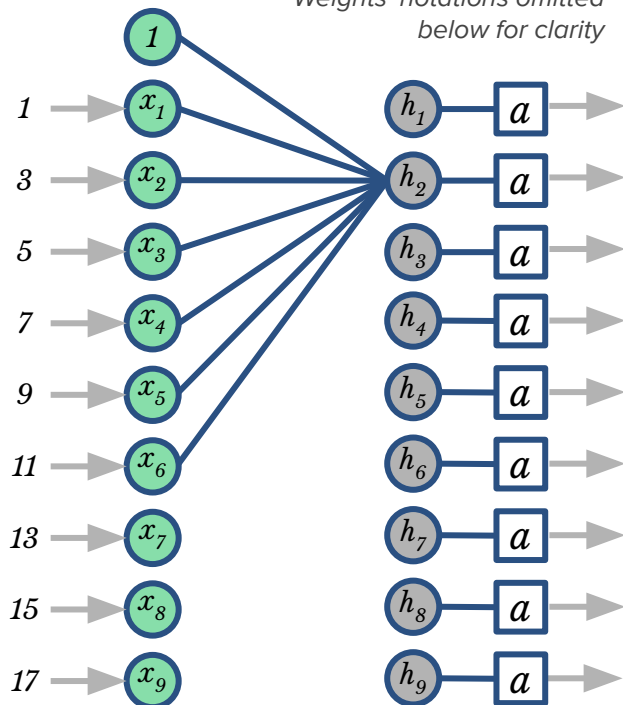
- Convolutions can be implemented as a NN layer, called **Convolutional Layer (ConvLayer)**.
- In it, we only need to zero many of the connections from its input to output (according to the convolution step) and keep the weights the same across units.

Convolution Operation



Convolutional Layer

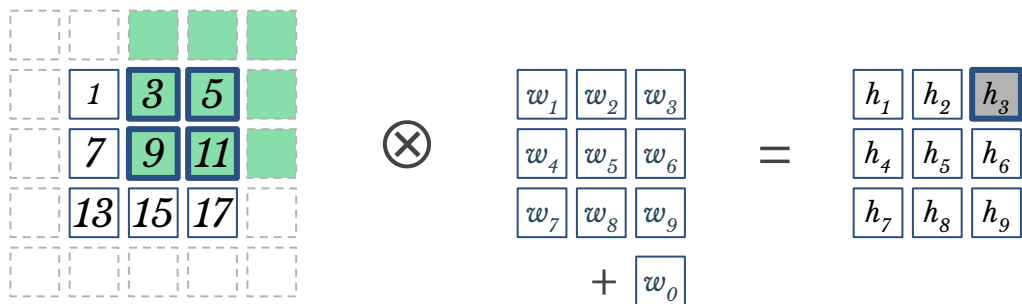
Weights' notations omitted below for clarity



Convolution as a Neural Network Layer

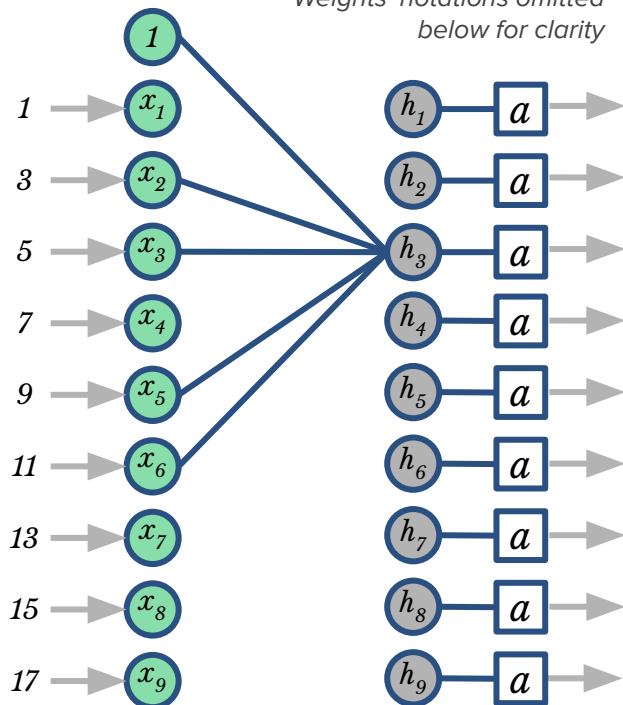
- Convolutions can be implemented as a NN layer, called **Convolutional Layer (ConvLayer)**.
- In it, we only need to zero many of the connections from its input to output (according to the convolution step) and keep the weights the same across units.

Convolution Operation



Convolutional Layer

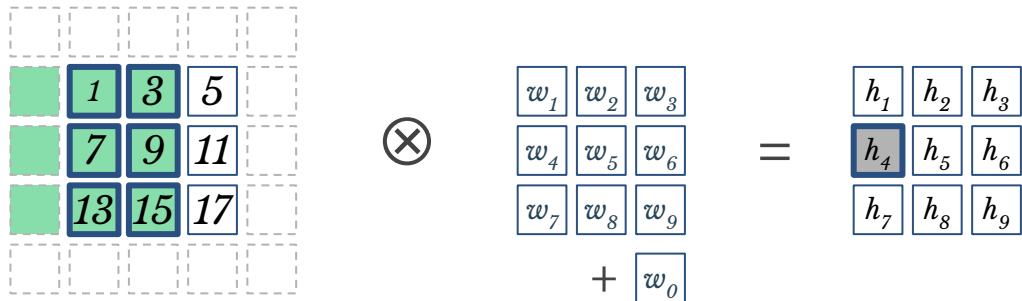
Weights' notations omitted below for clarity



Convolution as a Neural Network Layer

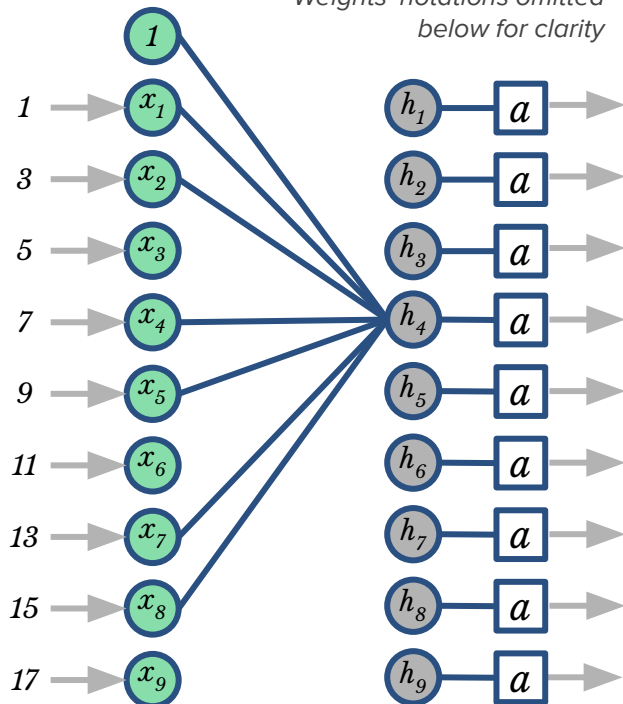
- Convolutions can be implemented as a NN layer, called **Convolutional Layer (ConvLayer)**.
- In it, we only need to zero many of the connections from its input to output (according to the convolution step) and keep the weights the same across units.

Convolution Operation



Convolutional Layer

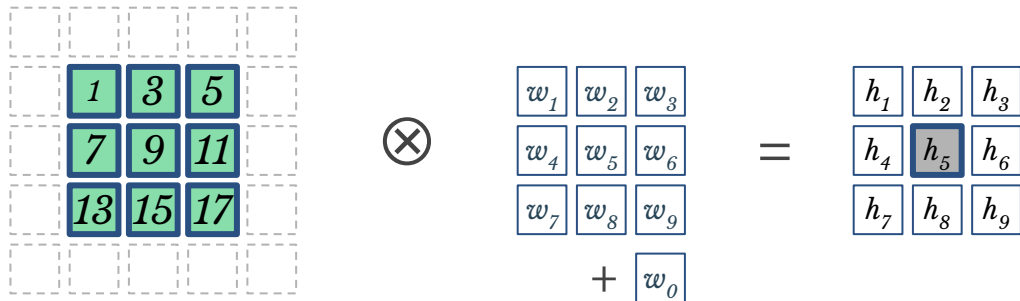
Weights' notations omitted below for clarity



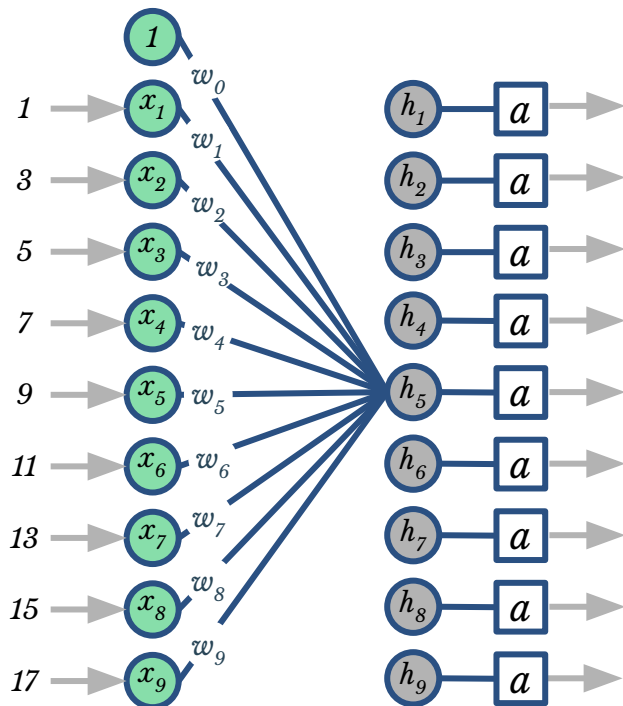
Convolution as a Neural Network Layer

- Convolutions can be implemented as a NN layer, called **Convolutional Layer (ConvLayer)**.
- In it, we only need to zero many of the connections from its input to output (according to the convolution step) and keep the weights the same across units.

Convolution Operation



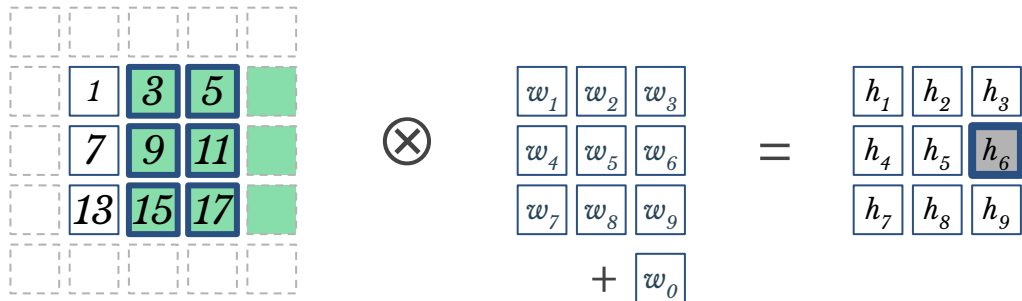
Convolutional Layer



Convolution as a Neural Network Layer

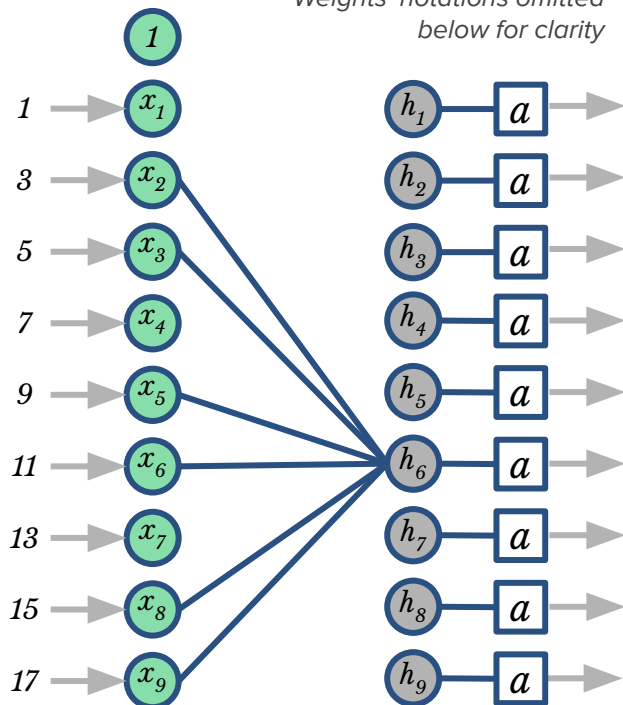
- Convolutions can be implemented as a NN layer, called **Convolutional Layer (ConvLayer)**.
- In it, we only need to zero many of the connections from its input to output (according to the convolution step) and keep the weights the same across units.

Convolution Operation



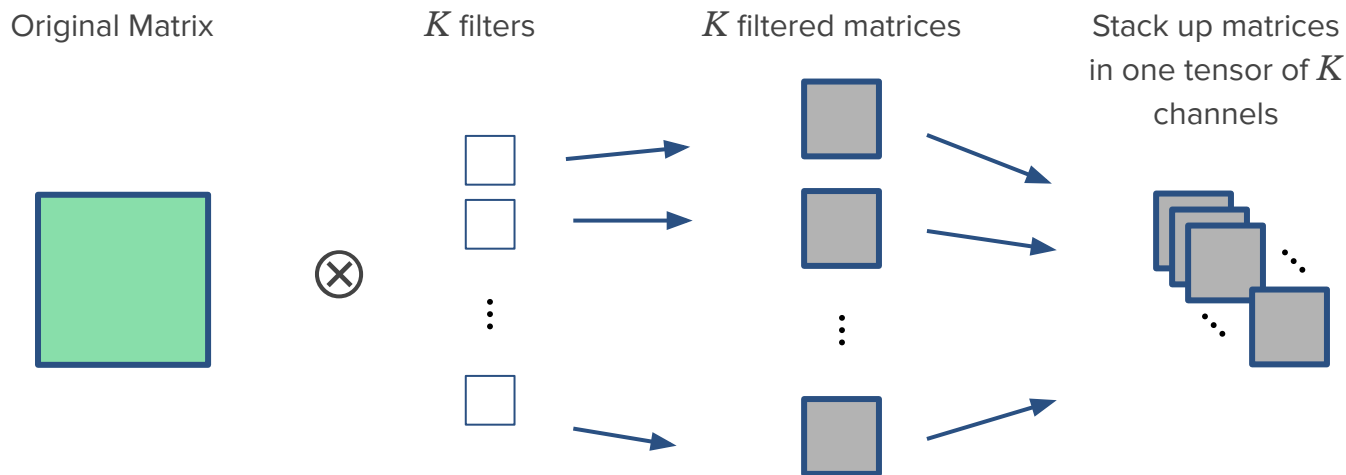
Convolutional Layer

Weights' notations omitted below for clarity



Multiple Filters

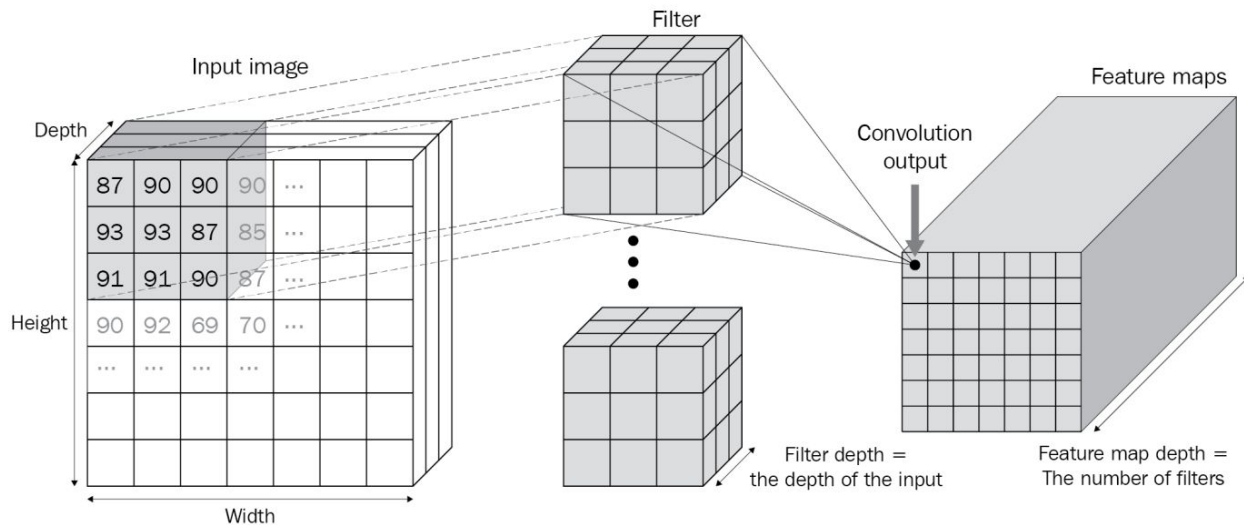
- Notice that we only had to learn **10 weights** in the convolutional layer from the last slide.
- In fact, that **wouldn't change** even for larger inputs, if we keep the same kernel size! This opens up the possibility of learning many filters without a big computation burden:



- Note that, in this process, we turned a matrix into a tensor of **multiple channels**.

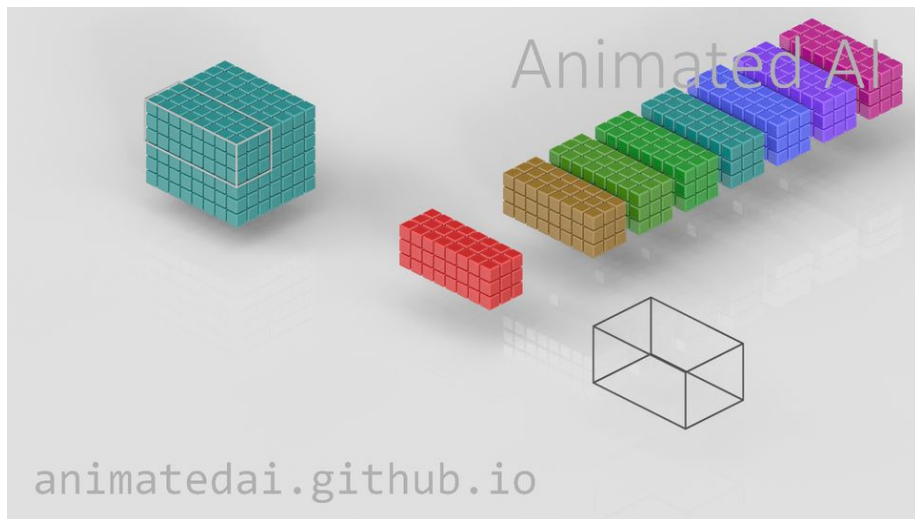
Convolution with Tensors

- We can generalize the convolution to images (or tensors in general) that have more than one channel.
- In that case, the kernels will have the same number of channels as the input image:



Convolution with Tensors

- We can generalize the convolution to images (or tensors in general) that have more than one channel.
- In that case, the kernels will have the same number of channels as the input image:

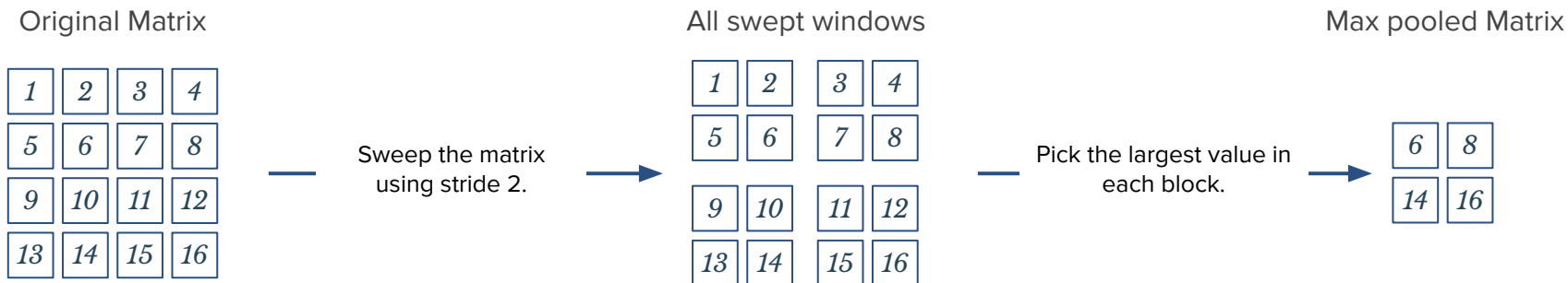


- In the example on the left*, we have:
 - An input tensor of shape $(8, 7, 6)$, i.e. 8 channels of size 7×6 .
 - A total of 8 filters/kernels, each of shape $(8, 3, 3)$.
 - An output tensor of shape $(8, 5, 4)$.
- Each channel in of the output tensor (represented by the color of its correspondent filter) is called a feature map in deep learning lingo.

* Check out this animation and more in this [link](https://animatedai.github.io).

The Max-pooling Operation

- Another important operation and layer in Deep Learning is called **Pooling (layer)**.
- The idea is to sweep the initial matrix as in a convolution, but now you apply a standard **not-learnable** operation for each window. The common operations are*:
 - **Average-pooling**: take the mean of the values in each window,
 - **Max-pooling**: pick the maximum of each window (by far the most used pooling type.)
- In the following, you have an example of the **2×2 Max-pooling with stride 2**:



- Max-pool layers are used to downscale the input by extracting the most important feature.

* [Here's](#) another good resource for understanding pooling and convolutional layers.

Exercise (*In pairs*)

- Using the two matrices on the right, perform the following operations:
 - Perform a 2×2 Max-pooling with stride 2 on A to create C .
 - Add 2 padding on C and compute its convolution with B to create D .
 - Compute the convolution of D with B as a kernel with stride 3.

Matrix A

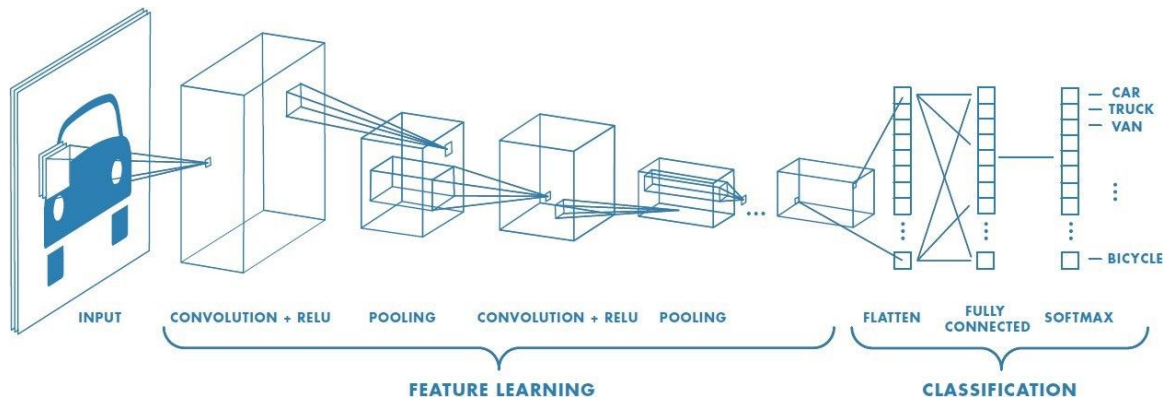
3	2	2	4
5	1	7	8
5	2	1	2
1	2	1	2

Matrix B

1	1	1
1	-2	1
1	1	1

Convolutional Neural Networks

- With convolutional and pooling layers, we create a **Convolutional Neural Network (CNN)**.
- In image classification tasks, these layers are used as a **feature learning step**, as they
 - Extract relevant features (convolutional layers, *more on it later in the course*)
 - Aggregate information (pooling layers).
- After the features are learned, we can flatten them and proceed with our usual Multilayer Perceptron (a series of **dense/linear layers**) for classification:



Creating a CNN in PyTorch

- In the same way that PyTorch offers `nn.Linear()` to define dense layers, it defines the module `nn.Conv2d()` to define convolutional layers:

```
nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, bias=True)
```

- Here's what these parameters mean*:
 - **in_channels** and **out_channels**: the number of channels (not the size of each datapoint) of the input and output. For example, if the inputs are grayscale images, and you want to use 32 filters in that layer, `in_channels = 1` and `out_channels = 32`.
 - **kernel_size**: the size of the kernel used in all filters of that layer. Inputting “3” here will give you kernels of size 3×3 . All kernels in a given layer have the same size.
 - **stride** and **padding**: sets the number of stride and the padding “thickness”. By default, the convolutions will sweep all the possible windows of the unpadded input.
 - **bias**: whether you wish to add a bias term (w_0 in [this slide](#)).

*Check the documentation [here](#) for more details on the layer and on other possible parameters.

Creating a CNN in PyTorch

- Next, we use PyTorch's `nn.MaxPool2d()` module to define a Max-pooling layer*:

```
torch.nn.MaxPool2d(kernel_size, stride=None, padding=0)
```

- The kernel size and padding parameters work exactly like in the `nn.Conv2d()` module.
- The stride parameter, however, is set to be of **the same size as the kernel size** if `stride=None`.
- In PyTorch, to flatten the output of a ConvLayer to prepare it for the Linear layers, we can use two approaches:
 - If you are defining a class that inherits `nn.Module`, you can use `torch.view()` in its `forward()` method to reshape the output of the ConvLayer.
 - When using `nn.Sequential()` to define a network, we have to add a `nn.Flatten()` module after the layer you want to flatten.

*Check the documentation [here](#) for more details on the layer and on other possible parameters.

Creating the CNN in PyTorch

- We define the following CNN:

```
model = nn.Sequential(nn.Conv2d( 1,64,kernel_size=3),
    nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Conv2d(64, 128, kernel_size=3),
    nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Flatten(),
    nn.Linear(3200, 200),
    nn.ReLU(),
    nn.Linear(200, 10)
).to(device)
```

- It has only two ConvLayers, that learn **64** and **128** filters, resp., each followed by a **2×2** Max-pool layer.
- The CNN part is then followed by a dense layer with **200** units.

- If we print our CNN's summary we get:

```
from torchsummary import summary
summary(model, (1, 28, 28)) # Notice the new shape
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 26, 26]	640
ReLU-2	[-1, 64, 26, 26]	0
MaxPool2d-3	[-1, 64, 13, 13]	0
Conv2d-4	[-1, 128, 11, 11]	73,856
ReLU-5	[-1, 128, 11, 11]	0
MaxPool2d-6	[-1, 128, 5, 5]	0
Flatten-7	[-1, 3200]	0
Linear-8	[-1, 200]	640,200
ReLU-9	[-1, 200]	0
Linear-10	[-1, 10]	2,010

=====
Total params: 716,706
Trainable params: **716,706** ← Notice how many weights to learn!
Non-trainable params: 0
===== (...)

Changing the dataset class

- Because we want to use the images in their original shapes, we don't need to flatten them when defining the Dataset. So our new class definition to (compare it to the original):

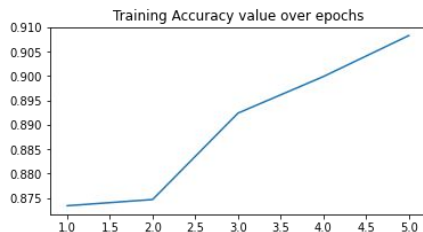
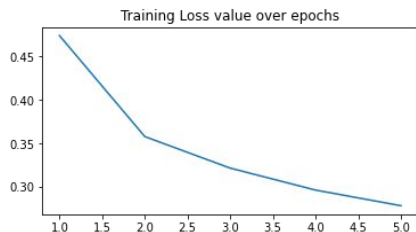
```
class FMNISTDataset(Dataset):  
    def __init__(self, x, y):  
        x = x.view(-1, 1, 28, 28)  
        x = x.float()/255  
        self.x, self.y = x, y  
    def __getitem__(self, ix):  
        return self.x[ix].to(device), self.y[ix].to(device)  
    def __len__(self):  
        return len(self.x)
```

- Note that we explicitly show that each datapoint has one channel (since they are grayscale images) and size 28×28 .
- Defining the data to show how many channels it has **is necessary** when using ConvLayers in the beginning of your model.

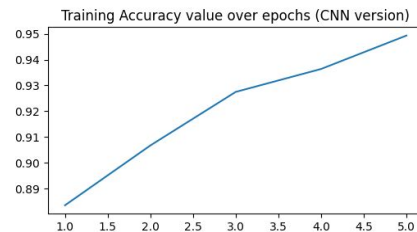
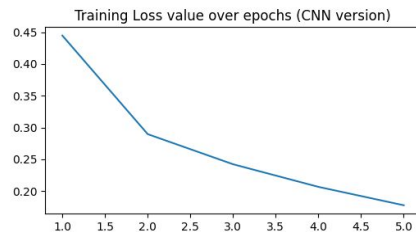
Training and testing the CNN

- Now, we can train our Convolutional Neural Network under the exact same parameters as the Multilayer Perceptron of the [previous slides](#) and compare their performance:

MLP – n. weights: 795k training time: 43s



CNN – n. weights: 716k, training time: 54s



- If we test our learned CNN, we also see the improvement compared to the [MLP one](#):

Test accuracy: 0.9146365523338318

- In sum:* with fewer weights to be learned, we were able to improve our results by using ConvLayers and Max-pooling, all thanks to the magic of **feature learning**!

Exercise (*In pairs*)

[Click here to open code in Colab](#)



- On the right you have the summary of today's CNN.

Explain:

- Why does the output shape progresses like depicted?
- Why does each layer have the number of weights (parameters) it says it has.

Here, the input shape is $(1, 28, 28)$ and the convolutions have 3×3 kernels.

Layer (type)	Output Shape	Param #
Conv2d-1	$[-1, 64, 26, 26]$	640
ReLU-2	$[-1, 64, 26, 26]$	0
MaxPool2d-3	$[-1, 64, 13, 13]$	0
Conv2d-4	$[-1, 128, 11, 11]$	73,856
ReLU-5	$[-1, 128, 11, 11]$	0
MaxPool2d-6	$[-1, 128, 5, 5]$	0
Flatten-7	$[-1, 3200]$	0
Linear-8	$[-1, 200]$	640,200
ReLU-9	$[-1, 200]$	0
Linear-10	$[-1, 10]$	2,010
Total params: 716,706		
Trainable params: 716,706		
Non-trainable params: 0		
(...)		

Video: color perception across cultures

